视频链路基座

从零手搓 Step1 教程

2026 FPGA 创新设计赛道（安路）赛题二

视觉方向 • Step1_video_link

读者：零基础队员
目标：理解并手搓视觉链路第一部分（Step1）
前置要求：会开电脑、装了 ModelSim 和 TD
预计学习时间：10～14 天（每天 2～3 小时）

目录

1 你在造什么 **4**
1.1 一句话 . . . 4
1.2 在整个系统中的位置 . . . 4
1.3 为什么要先做这个 . . . 4

2 阶段 0：三个数字电路概念（1～2 天） **4**
2.1 时钟（clock） . . . 4
2.2 寄存器（D 触发器） . . . 4
2.3 组合逻辑 . . . 5
2.4 练习：写你的第一个 Verilog 计数器 . . . 5

3 阶段 1：Verilog 最小语法集（3～5 天） **6**
3.1 必须会的 8 个语法 . . . 6
3.2 三条铁律（违反 = 仿真对但上板错） . . . 7
3.3 练习：读懂本项目的一个模块 . . . 8

4 阶段 2：视频流概念（2～3 天） **8**
4.1 像素流长什么样 . . . 8
4.2 三个” 路标” 信号——全项目的通用语言 . . . 8
4.3 一帧的完整结构 . . . 8
4.4 96bit 打包：4 像素一拍 . . . 8
4.5 练习：跑仿真看波形 . . . 9

5 阶段 3：精读 ser96to24——你的第一个模块（3～5 天） **9**
5.1 它做什么（生活类比） . . . 9
5.2 为什么要这么做 . . . 9
5.3 打开代码逐段读 . . . 9
5.4 第 1 段：端口声明 . . . 9
5.5 第 2 段：小仓库（FIFO） . . . 10
5.6 第 3 段：写入逻辑 . . . 10
5.7 第 4 段：读出逻辑 . . . 10
5.8 第 5 段：为什么不会溢出 . . . 11
5.9 练习 . . . 11

6 阶段 4：读懂 tpg_720p——测试图案发生器（2～3 天） **11**
6.1 它做什么 . . . 11
6.2 它长什么样 . . . 11
6.3 为什么竖条要” 避开红色” . . . 11
6.4 精读要点 . . . 11
6.5 练习 . . . 12

7 阶段 5：CDC 概念（可选，2～3 天） 12
7.1 什么是跨时钟域 . . . 12
7.2 两根” 安全桥” . . . 12
7.3 学习建议 . . . 12

8 完整验证：跑通 Step1 全部测试 12
8.1 一键回归 . . . 13
8.2 单跑某个仿真 . . . 13

第一部分 工程设计详解：Step1 的实现分析 13

9 工程总览 13
9.1 文件清单 . . . 13
9.2 数据流 . . . 14
9.3 三条设计原则 . . . 14

10 模块详解：ser96to24 14
10.1 为什么需要它 . . . 15
10.2 核心结构：FIFO . . . 15
10.3 写入逻辑（4 像素一次性入队） . . . 15
10.4 读出逻辑（每拍 1 像素） . . . 16
10.5 计数器更新（同时读写） . . . 16
10.6 溢出保护（仅仿真） . . . 16

11 模块详解：tpg_720p 17
11.1 三个用途 . . . 17
11.2 图案设计：为什么是这 8 个颜色 . . . 17
11.3 几何计数器与颜色函数 . . . 17
11.4 4 分频节拍（关键设计） . . . 18
11.5 帧同步标志 . . . 18
11.6 LFSR 随机空闲（可选，锻炼鲁棒性） . . . 19

12 模块详解：pulse_sync（单 bit 脉冲跨域） 19
12.1 问题 . . . 19
12.2 解决方案：toggle + 边沿还原 . . . 19

13 模块详解：cdc_4phase（多 bit 数据跨域） 19
13.1 问题 . . . 19
13.2 解决方案：4 相握手 . . . 20
13.3 关键实现与一个真实 bug . . . 20
13.4 验证 . . . 20

14 仿真专用模块 **21**
14.1 sim_video_src（文件回放源） 21
14.2 frame_dump24（帧转储） 21

15 测试平台设计 **21**
15.1 四层验证策略 21
15.2 tb_ser96to24 的关键设计：唯一数据编码 21
15.3 tb_tpg 的双重验证 22
15.4 tb_cdc4phase 的时序陷阱 22

16 验证结果 **22**
16.1 回归输出 22
16.2 Step1 四项测试细节 23
16.3 Step1 阶段抓到的 5 个真缺陷 23

17 上板挂载说明 **23**
17.1 在官方例程中的位置 23
17.2 挂载步骤（给上板组） 24
17.3 资源占用 24

18 总结与下一步 **24**
18.1 Step1 学到了什么 24
18.2 下一步学什么 24

1 你在造什么

1.1 一句话

把摄像头拍到的像素” 接住”，把测试图案” 生成” 出来，把不同时钟的信号” 安全传递”。
翻译成电路：计数器、比较器、小存储器、几根线。没有复杂数学，没有算法——这是整个项目里最简单的一步，也是最好的入门点。

1.2 在整个系统中的位置

摄像头拍到的画面要经过好几级处理才能到屏幕上。Step1 是这条路的” 地基”——它不做人脸识别、不做跟踪，它只保证**像素数据能从 A 点流到 B 点而不丢、不错、不停**。

Step1 的三个模块

1. **ser96to24**：把摄像头送来的”4 像素一包” 拆成”1 像素一拍”（串化器）
2. **tpg_720p**：没有摄像头时，假装自己是摄像头，生成测试图案（测试源）
3. **pulse_sync + cdc_4phase**：把信号安全地从一个时钟域送到另一个时钟域（跨域桥）

1.3 为什么要先做这个

后面的检测链（Step2：颜色识别）、闭环控制（Step3：云台转动）、OSD 叠加（Step4：屏幕画框），全都建立在这条” 像素流” 之上。如果像素流本身就有问题（丢数据、顺序错、帧同步乱），后面全部白搭。

学完 Step1 你能做到

- 解释像素流从摄像头到屏幕的完整路径
- 看懂 Verilog 里的 always/assign/reg/wire
- 在 ModelSim 里跑仿真并看波形
- 手写一个计数器、一个串化器、一个测试图案发生器

2 阶段 0：三个数字电路概念（1～2 天）

你不需要学完整门数电课。只需要理解三个东西，就能看懂本项目 90% 的代码。

2.1 时钟（clock）

一个不停 0→1→0→1 的方波信号。板上有个晶振每秒产生 5000 万个这样的方波（50MHz）。
关键点：所有动作都在时钟” 从 0 变 1 的那一瞬间” 发生。就像军队的口令——喊” 齐步走” 的瞬间所有人同时迈步。时钟就是那个口令。

2.2 寄存器（D 触发器）

一个” 在时钟上升沿锁存输入值的 1-bit 存储单元”。



```
输入 d:   1    0    1    1    ...
输出 q:   旧值  1    0    1    ...
               ↑    ↑    ↑
     q 在每个上升沿" 记住" 当时的 d
```

Verilog 写法：

```
always @(posedge clk) begin
    q <= d;     // 每个时钟上升沿，q 取 d 的值
end
```

2.3 组合逻辑

没有存储、没有时钟——输入变了输出**立刻**变（有微小延迟）。例如加法器、比较器、多路选择器。

```
wire [7:0] sum;
assign sum = a + b;     // b 变了 sum 立刻跟着变（几纳秒后）
```

组合逻辑 vs 时序逻辑——一字之差，天壤之别

时序逻辑（有时钟，用 `always @(posedge clk)`）：输出在时钟沿才变，有”记忆”。

组合逻辑（无时钟，用 `assign`）：输出立刻跟输入变，无记忆。

混淆的后果：仿真可能对但上板行为完全不同——这是新手最常踩的坑。

2.4 练习：写你的第一个 Verilog 计数器

```
// my_counter.v ———一个 4 位计数器，0~15 循环
module my_counter (
    input  wire       clk,      // 时钟
    input  wire       rst_n,    // 复位（低电平有效）
    output reg  [3:0] count     // 4 位计数输出
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            count <= 4'd0;           // 复位时清零
        else
            count <= count + 4'd1;   // 每拍加 1
    end
endmodule
```

怎么跑仿真

1. 创建一个文件 my_counter.v，粘贴上面的代码
2. 再创建一个测试文件 tb.v：

```
module tb;
    reg clk = 0, rst_n = 0;
    always #5 clk = ~clk;          // 每 5ns 翻转 = 100MHz
    my_counter u (.clk(clk), .rst_n(rst_n), .count());
    initial begin
        #20 rst_n = 1;             // 20ns 后释放复位
        #200 $finish;              // 跑 200ns
    end
endmodule
```

3. 在命令行：

```
vlib work
vlog my_counter.v tb.v
vsim -c tb -do "run -all; quit -f"
```

4. 或用 GUI：vsim tb → 点 Run → 加波形看 count 从 0 数到 15

3 阶段 1：Verilog 最小语法集（3～5 天）

你不需要学完整 Verilog。以下是你看懂本项目所需的全部语法，大约 30 分钟过一遍。

3.1 必须会的 8 个语法

```
// 1. 模块声明（一个电路块）
module 模块名 #(参数) (端口列表);
  ...
endmodule

// 2. 端口类型
input  wire [7:0]  data_in    // 8 位输入
output reg  [23:0] data_out   // 24 位输出（reg 表示在 always 里赋值）
output wire        ready      // 1 位输出（wire 表示在 assign 里赋值）
inout  wire        sda        // 双向（如 I2C）

// 3. 时序逻辑（有时钟）
always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        q <= 0;
    else
        q <= d;
end

// 4. 组合逻辑
assign sum = a + b;

```

```
// 5. always 组合逻辑（多个条件时用这个代替多条 assign）
always @(*) begin
    if (sel) result = a;
    else     result = b;
end

// 6. 位拼接（把多个信号拼在一起）
wire [15:0] combined = {high_byte, low_byte};   // {8bit, 8bit} = 16bit

// 7. 位选择
wire bit3 = combined[3];          // 取第 3 位
wire [3:0] nibble = combined[7:4]; // 取高 4 位

// 8. 常量写法
8'd42     // 8 位十进制 42
8'h2A     // 8 位十六进制 2A
4'b1010   // 4 位二进制 1010
```

3.2 三条铁律（违反 = 仿真对但上板错）

铁律 1：时序逻辑用 <=，组合逻辑用 =

```
// 正确
always @(posedge clk) q <= d;          // 时序：<=
assign sum = a + b;                     // 组合：不需要符号

// 错误
always @(posedge clk) q = d;           // 时序用了 = → 竞争冒险
```

铁律 2：每个寄存器都要有复位值

```
// 正确：复位时给确定的值
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) count <= 0;            // ← 复位值
    else        count <= count + 1;
end

// 错误：没有复位 → 上电后寄存器是 X（未知态）
// 本项目实际踩过：tpg 的 pres 计数器漏了复位 → 输出全是 X
```

铁律 3：有符号运算必须显式声明

```
wire signed [12:0] err = $signed(cx) - $signed(center);
// 缺 $signed → 当无符号 → 负数变成巨大正数 → 系统跑飞
// 本项目实际踩过：闭环模型跑飞，查了半天是无符号减法
```

3.3 练习：读懂本项目的一个模块

打开 `03_vision/step1/03_project/rtl/tpg_720p.v`，找到以下结构并在旁边标注它们对应上面 8 个语法中的哪一个：

- 模块声明和参数（第 1～10 行）
- 内部寄存器声明（`beat_x`, `y`, `pres`）
- 颜色生成函数（`px_color`）
- 主时序逻辑块（`always @(posedge I_clk ...)`）
- 输出赋值（`O_tdata <= ...`）

4 阶段 2：视频流概念（2～3 天）

4.1 像素流长什么样

想象一根水管，每秒流出 5500 万个像素。每个像素是 24 位（R 8bit + G 8bit + B 8bit）。像素排队通过，不能停、不能丢。

4.2 三个”路标”信号——全项目的通用语言

valid / sof / eol——必须吃透

- **valid**：这拍的数据是真的（不是垃圾）。类比：传送带上有东西。
- **sof**（Start of Frame）：这拍是**一帧的第一个像素**（左上角）。类比：新的一页纸开始了。
- **eol**（End of Line）：这拍是**一行的最后一个像素**（最右边）。类比：这行写完了，换行。

4.3 一帧的完整结构

```
tuser=1, px(0,0), px(1,0), ..., px(1279,0), tlast=1   ← 第 0 行（320 拍）
tuser=0, px(0,1), px(1,1), ..., px(1279,1), tlast=1   ← 第 1 行
...
tuser=0, px(0,719), ..., px(1279,719), tlast=1        ← 第 719 行

一帧 = 1280 × 720 = 921,600 个像素
帧率 60fps → 每秒 55,296,000 个像素流过
```

坐标系约定

- 原点 (0,0) 在**左上角**
- x 向右增长（0..1279）
- y 向下增长（0..719）
- 这与 LCD/HDMI 扫描方向一致

4.4 96bit 打包：4 像素一拍

为了减少布线，像素不是 1 个 1 个传的，而是 4 个打包成一拍：

```
I_tdata[95:72]  = 像素 0（最左边）  {R[7:0], G[7:0], B[7:0]}
I_tdata[71:48]  = 像素 1
I_tdata[47:24]  = 像素 2
I_tdata[23:0]   = 像素 3（最右边）
```

```
一帧 1280 像素 ÷ 4 = 320 拍/行
一帧 = 320 拍 × 720 行 = 230,400 拍
```

4.5 练习：跑仿真看波形

1. 在 ModelSim 里跑 `tb_tpg`
2. 加波形：`add wave /tb_tpg/u_dut/*`
3. 跑 1 帧的时间（约 13ms），找：
   - `tuser` 什么时候变高？（应该在帧的最开头，只出现一次）
   - `tlast` 什么时候变高？（应该每 320 拍出现一次）
   - `tdata` 在变化吗？（应该在不同竖条之间切换）

5 阶段 3：精读 ser96to24——你的第一个模块（3～5 天）

5.1 它做什么（生活类比）

- **输入**：一辆卡车每 4 秒来一次，装 4 个箱子（96bit = 4 × 24bit 像素）
- **输出**：每秒送走 1 个箱子（24bit 像素）
- **本质**：一个”小仓库”（FIFO），卡车来了卸 4 个进去，出口每秒取 1 个

5.2 为什么要这么做

后面的处理链（HSV/阈值/平滑）每个时钟周期只处理 1 个像素。如果不拆，就要写 4 份处理逻辑——资源翻 4 倍，调试难度翻 10 倍。

5.3 打开代码逐段读

文件：`03_vision/step1/03_project/rtl/ser96to24.v`（约 80 行）

5.4 第 1 段：端口声明

```
module ser96to24 (
    input  wire        I_clk,       // 时钟
    input  wire        I_rst_n,     // 复位
    input  wire [95:0] I_tdata,     // 输入：4 像素打包
    input  wire        I_tvalid,    // 输入有效
    input  wire        I_tuser,     // 帧起始标记
    input  wire        I_tlast,     // 行尾标记
    output wire        O_tready,    // 永远 =1（" 我随时能收"）
    output reg         O_valid,     // 输出有效
    output reg         O_sof,       // 输出帧起始
    output reg         O_eol,       // 输出行尾
    output reg  [23:0] O_data       // 输出：1 像素
);
```

O_tready 为什么恒 1？

这是”窃流”语义——视觉分支是旁路，不影响主链路。上游爱发多少发多少，我们只管”听”。

5.5 第 2 段：小仓库（FIFO）

```
reg [23:0] mem [0:7];      // 8 个格子，每个 24bit
reg [2:0]  wp;              // 写指针（指向下一个空格子）
reg [2:0]  rp;              // 读指针（指向最老的格子）
reg [3:0]  cnt;             // 里面有几个
```

FIFO 的本质

FIFO = First In, First Out（先进先出）。就像排队买奶茶：先来的人先买到。写指针往队尾加人，读指针从队头取人。

5.6 第 3 段：写入逻辑

```
wire do_wr = I_tvalid;      // 输入有效就写入

always @(posedge I_clk) begin
    if (do_wr) begin
        mem[wp]     <= I_tdata[95:72];  // 像素 0
        mem[wp_n]   <= I_tdata[71:48];  // 像素 1
        mem[wp_n2]  <= I_tdata[47:24];  // 像素 2
        mem[wp_n3]  <= I_tdata[23:0];   // 像素 3
        wp <= wp + 4;                    // 写指针跳 4 格
        cnt <= cnt + 4;                  // 计数 +4
    end
end
```

5.7 第 4 段：读出逻辑

```
wire do_rd = (cnt != 0);    // 里面有数据就读

always @(posedge I_clk) begin
    if (do_rd) begin
        O_valid <= 1;
        O_data  <= mem[rp];    // 取最老的
        O_sof   <= sof_q[rp];  // 如果这是帧首像素
        O_eol   <= eol_q[rp];  // 如果这是行尾像素
        rp <= rp + 1;
        cnt <= cnt - 1;
    end else begin
        O_valid <= 0;
    end
end
```

5.8 第 5 段：为什么不会溢出

收支平衡

- 输入：每 4 拍来 1 个 beat（含 4 像素）→ 平均 1 像素/拍
- 输出：每拍最多读 1 像素 → 1 像素/拍
- 平衡！缓冲峰值 7 < 深度 8，不会溢出
- 如果输入全速（每拍 1 beat），缓冲会爆——所以源端必须按 1/4 占空发数据

5.9 练习

1. 画出 ser96to24 的框图（输入 → FIFO → 输出）
2. 在 ModelSim 跑 `tb_ser96to24`，看波形确认：
   - 输入 4 像素一拍 → 输出 1 像素一拍
   - sof 出现在帧的第一个像素
   - eol 出现在行的最后一个像素
3. 尝试：把 FIFO 深度从 8 改成 16（`mem [0:15]`，指针改 4bit），重新仿真

6 阶段 4：读懂 tpg_720p——测试图案发生器（2～3 天）

6.1 它做什么

没有摄像头时，TPG 假装自己是摄像头。它内部就是几个计数器驱动一个” 查色表”：

```
x 计数器：0→1279 循环（列位置）
y 计数器：0→719 循环（行位置）
颜色 = f(x, y)  ← 一个函数，根据位置返回颜色
```

6.2 它长什么样

```
┌──────────────────────────────────────────────┐
│ 灰 1 │ 灰 2 │ 灰 3 │ 灰 4 │ 蓝 │ 绿 │ 深绿 │ 深蓝 │
│      │      │      │      │    │    │      │      │
│  ┌──────────┐                                │
│  │  红色方块  │ ← 这是目标跟踪的" 目标"        │
│  └──────────┘                                │
│      （方块位置由外部输入控制，可以移动）      │
└──────────────────────────────────────────────┘
    8 个竖条各 160px 宽，全部避开红色阈值
```

6.3 为什么竖条要” 避开红色”

因为后面的检测链在找红色目标。如果背景条也有红色，检测就会误识别。所以背景色选了灰阶（无饱和度）、蓝色（色相 240）、绿色（色相 120）——都不是红色（色相 0）。

6.4 精读要点

```
1 // 核心：一个函数根据 x 坐标返回颜色
2 function [23:0] px_color;
3     if (在方块内)      px_color = 24'hE0_20_20;  // 红色
```

```
    else if (px < 160)  px_color = 24'h40_40_40;  // 灰条 1
    else if (px < 320)  px_color = 24'h80_80_80;  // 灰条 2
    ...
endfunction

// 每拍输出 4 个像素（一个 beat）
O_tdata <= {px_color(x×4+0, y), px_color(x×4+1, y),
            px_color(x×4+2, y), px_color(x×4+3, y)};
```

6.5 练习

1. 改一种条的颜色（如把蓝色改成紫色）→ 重新仿真 → 确认输出变化
2. 改方块位置（`I_box_x` 和 `I_box_y`）→ 确认方块移动
3. 思考：如果把一个条改成红色（E02020），检测链会怎么反应？

7 阶段 5：CDC 概念（可选，2～3 天）

7.1 什么是跨时钟域

芯片里不同模块可能用不同的时钟。比如：摄像头数据用 74.25MHz，舵机控制用 50MHz。当一个信号从 74.25MHz 的模块传到 50MHz 的模块时，就会出问题——因为接收方可能在信号变化的”一半”时采样到不确定的值。

为什么不能直接连线

如果 74.25MHz 信号在 50MHz 时钟上升沿的**同一时刻**变化，接收寄存器可能进入”亚稳态”（既不是 0 也不是 1），后续电路全部出错。

7.2 两根”安全桥”

pulse_sync（单 bit 脉冲跨域，12 行）

原理：源域每发一个脉冲，翻转一个寄存器。目标域用两个寄存器同步这个翻转，再检测”变化”产生脉冲。

cdc_4phase（多 bit 数据跨域，80 行）

原理：源域把数据放在总线上 + 翻转一个 req 信号。目标域发现 req 变化后，锁存数据，翻转 ack 回应。源域收到 ack 后可以发下一个。

就像寄快递：你把包裹放门口（数据 +req），对方取走后放个签收回执（ack），你看到回执才放下一个包裹。

7.3 学习建议

先跳过这两个文件（不影响理解其他模块），等学完 Step2 的检测链后再回来看。只需要记住一条规则：**跨时钟域不能裸连，必须走这两个模块。**

8 完整验证：跑通 Step1 全部测试

8.1 一键回归

```
cd D:\fpga-ptz-tracker\tools
python run_all.py
```

期望输出：`汇总：18/18 PASS`（仓库全量口径；本册阶段为 15 项，其后 Step5/闭环补强又新增 3 项）

其中 Step1 相关的 4 个测试：

- `tb_ser96to24`：随机数据 + 3 帧，队列自检（数据/sof/eol 逐像素）
- `tb_tpg`：几何计数断言 + 整帧 921600 像素与金标准比对
- `tb_pulse_sync`：异步双钟 100 脉冲 1:1
- `tb_cdc4phase`：异步双钟 200 随机数据完整性

8.2 单跑某个仿真

```
cd 03_vision/step1_video_link/03_project/sim

# 编译
vlib work
vlog ../rtl/ser96to24.v ../tb/tb_ser96to24.v

# 跑仿真（命令行模式）
vsim -c tb_ser96to24 -do "run -all; quit -f"

# 跑仿真（GUI 模式，可以看波形）
vsim tb_ser96to24
# 在 GUI 里: add wave /*  →  Run All
```

第一部分　工程设计详解：Step1 的实现分析

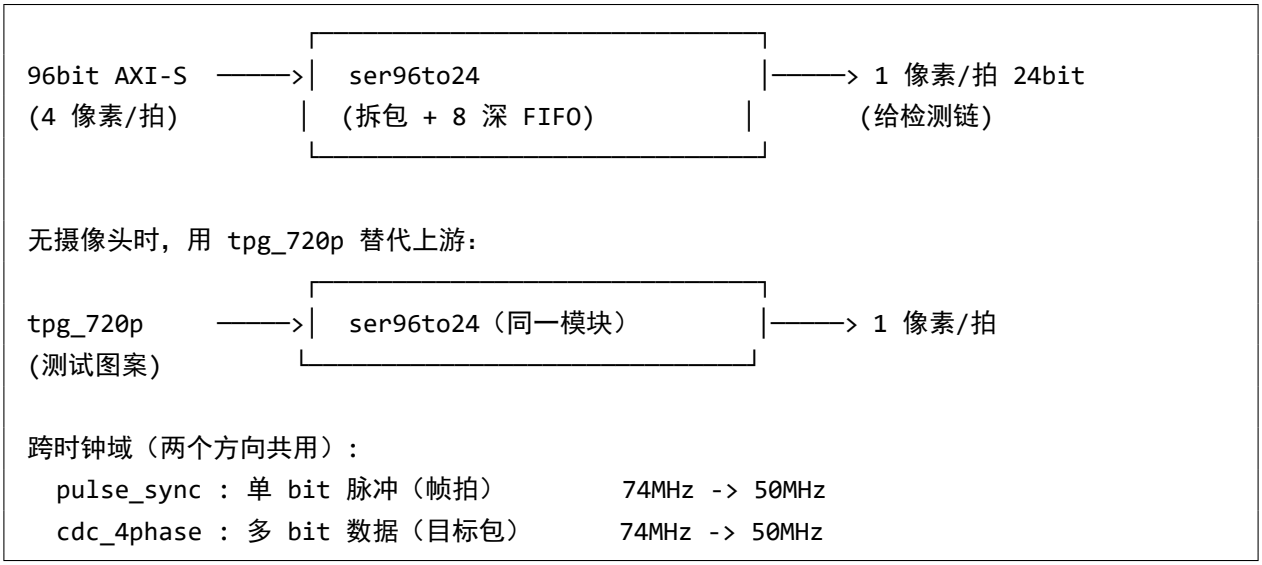
9 工程总览

9.1 文件清单

Step1 的全部工程文件位于 `03_vision/step1_video_link/03_project/`：

文件	行数	类型	作用
ser96to24.v	96	可综合	96bit 转 1 像素/拍串化器
tpg_720p.v	105	可综合	720p 测试图案发生器
pulse_sync.v	37	可综合	单 bit 脉冲跨时钟域
cdc_4phase.v	78	可综合	多 bit 数据 4 相握手跨域
sim_video_src.v	73	仅仿真	回放文件向量
frame_dump24.v	68	仅仿真	帧转储为 hex（供 Python 比对）
tb_ser96to24.v	90	测试	串化器 TB
tb_tpg.v	72	测试	TPG 几何断言 + 帧比对
tb_pulse_sync.v	45	测试	跨域脉冲 1:1
tb_cdc4phase.v	63	测试	跨域数据完整性

9.2 数据流



9.3 三条设计原则

贯穿全工程的设计原则

1. **窃流旁路**：视觉分支绝不反压主线（`O_tready` 恒 1），出错可随时摘除，显示链路行为与官方例程完全一致
2. **结构自对齐优先**：能用”读写同指针”解决对齐，就不用”移位 + 手工推相位”（后者在 Step2 的平滑模块上让团队返工了四版）
3. **金标准驱动**：所有算术先在 Python 建位精确模型，RTL 逐位复刻并回归验证

10 模块详解：ser96to24

10.1 为什么需要它

摄像头/ISP 输出的是 4 像素打包（96bit/拍），这样布线少、带宽高。但后续检测链（HSV 变换、阈值、平滑）每拍只能处理 1 个像素——如果按 4 像素写，就要 4 份并行逻辑：资源 ×4、调试难度 ×10。

设计决策：在链路入口做一次” 降速”，把 4 像素/拍拆成 1 像素/拍。

10.2 核心结构：FIFO

```
// 8 深缓冲：数据与标志同址存放
reg [23:0] mem   [0:7];      // 像素数据
reg        sof_q [0:7];      // 帧首标志
reg        eol_q [0:7];      // 行尾标志
reg [2:0]  wp, rp;           // 写指针 / 读指针
reg [3:0]  cnt;              // 当前存了几个
```

为什么深度是 8？ 输入每 4 拍来 4 像素（+4），输出每拍取 1 像素（−1）。最坏情况净积压 +3/拍；考虑行尾边界与读空闲，实测峰值深度 7。深度 8 留 1 格余量，且 8 是 2 的幂——指针用 3bit 自然回绕，无需取模。

10.3 写入逻辑（4 像素一次性入队）

```
wire do_wr = I_tvalid & O_tready;   // O_tready 恒 1，故等价于 I_tvalid

always @(posedge I_clk or negedge I_rst_n) begin
    if (!I_rst_n) begin
        wp <= 3'd0; rp <= 3'd0; cnt <= 4'd0;
    end else begin
        if (do_wr) begin
            mem[wp]      <= p0;          // 像素 0（最左）
            mem[wp_n]    <= p1;
            mem[wp_n2]   <= p2;
            mem[wp_n3]   <= p3;          // 像素 3（最右）
            sof_q[wp]    <= I_tuser;     // 帧首标志只挂第 1 个像素
            eol_q[wp_n3] <= I_tlast;     // 行尾标志只挂第 4 个像素
            wp <= wp + 4'd4;
        end
        ...
```

关键点 1：标志与像素严格绑定

`tuser` 表示” 这一拍是帧首”，但展开后真正的帧首只有 4 个像素中的**第 1 个**。所以 `sof_q[wp]` 只给第 1 格，其余 3 格给 0。`tlast` 同理，只给第 4 个像素。

这一步如果写错（比如给整拍 4 个像素都置位），下游的帧计数会变成 4 倍。

关键点 2：指针步进 4 一次写入占 4 格，所以 `wp += 4`。指针位宽 3bit（0~7），加 4 后自然回绕，无需额外逻辑。

10.4 读出逻辑（每拍 1 像素）

```
wire do_rd = (cnt != 4'd0);      // 有数据就读

always @(*) begin
    O_valid = do_rd;             // 组合输出
    O_data  = mem[rp];
    O_sof   = sof_q[rp];
    O_eol   = eol_q[rp];
end
```

为什么用组合读出而不是寄存输出？ 分布式 RAM 的组合读延迟很短（几个 LUT），在 74MHz 下时序满足。若改成寄存器输出，整个检测链会多延迟 1 拍，给帧内对齐带来额外复杂度。

10.5 计数器更新（同时读写）

```
case ({do_wr, do_rd})
    2'b10:   cnt <= cnt + 4'd4;    // 只写
    2'b01:   cnt <= cnt - 4'd1;    // 只读
    2'b11:   cnt <= cnt + 4'd3;    // 同时读写：+4-1 = +3
    default: cnt <= cnt;
endcase
```

新手最容易写错的地方

分开写两个 `if` 会出错：

```
if (do_wr) cnt <= cnt + 4;
if (do_rd) cnt <= cnt - 1;   // ← 错误！后一句覆盖前一句
```

同时发生时，后者把前者的结果覆盖掉，计数逐渐跑偏（每拍少加 4 或多减 1）。**必须用 case 把四种组合合并处理**——这是 FIFO 的标准写法。

10.6 溢出保护（仅仿真）

```
// synthesis translate_off
always @(posedge I_clk) begin
    if (I_rst_n && (cnt > 4'd7))
        $display("SER96TO24_FATAL: overflow cnt=%0d t=%0t", cnt, $time);
end
// synthesis translate_on
```

`translate_off/on` 之间的代码只在仿真时存在，综合时被完全忽略——既能第一时间暴露问题，又不占任何硬件资源。

这个保护立刻抓到了真 bug

首版仿真时 TB 和 TPG 都以**满速**发 beat（相当于 4 倍像素率），断言立刻报 `overflow cnt=13`。

根因：真实 AXI-S 总线上 beat 占空比是 1/4（beat 率 = 像素率/4），不是每拍都发。720p60 的像素率 55.3Mpx/s ÷ 4 = 13.8M beat/s，在 74.25MHz 下就是" 每 4 拍 1 个 beat"。

修正：TPG / TB / sim_video_src 统一加 4 分频节拍。这个 bug 如果留到上板，现象会是" 画面偶尔丢像素"，极难定位。

11 模块详解：tpg_720p

11.1 三个用途

1. **无摄像头演示**：板子上没插摄像头时也能出图，独立演示视觉链路
2. **闭环仿真驱动**：方块位置由外部输入控制，TB 可用云台模型实时改变它（相机在环）
3. **确定性测试源**：图案完全可预测，Python 能算出期望帧做逐像素比对

11.2 图案设计：为什么是这 8 个颜色

竖条	RGB	HSV（近似）	为什么选它
灰 1	404040	H=0, S=0, V=64	无饱和度，不触发任何颜色阈值
灰 2	808080	H=0, S=0, V=128	同上，验证中灰
灰 3	C0C0C0	H=0, S=0, V=192	同上，验证亮灰
灰 4	E0E0E0	H=0, S=0, V=224	高亮但无色——测" 高亮无饱和" 边界
蓝	2020E0	H=240	与红色（H=0）相差最远的色相
绿	20E020	H=120	三基色之一，验证中间色相
深绿	106010	H=120, V=96	低亮度绿色——测暗色
深蓝	101060	H=240, V=96	低亮度蓝色——测暗色
红方块	E02020	H=0, S=192, V=224	**唯一目标**，高饱和高亮度红

设计要点：所有背景色都避开了红色阈值窗（$\mathbf{H}\in[335,25]$ 且 $\mathbf{S}\geq 80$ 且 $\mathbf{V}\geq 60$）。 灰阶 S=0 不满足饱和度条件；蓝色 H=240、绿色 H=120 都不在色相窗内。这样检测链一眼锁定红方块，不受背景干扰——这是后面" 逐像素金标准比对" 能通过的前提。

11.3 几何计数器与颜色函数

```
localparam BEATS_PER_LINE = IMG_WIDTH / 4;   // 1280/4 = 320

reg [10:0] beat_x;   // 0..319（打包坐标）
reg [9:0]  y;        // 0..719（行坐标）
reg [1:0]  pres;     // 4 分频节拍

function [23:0] px_color;
    input [12:0] px;    // 真实像素列号
    input [9:0]  py;
```

```
    begin
        if (BOX_EN && (px >= I_box_x) && (px < I_box_x + BOX_W) &&
            (py >= I_box_y) && (py < I_box_y + BOX_H))
            px_color = 24'hE0_20_20;              // 红方块
        else if (px < 13'd160)  px_color = 24'h40_40_40;
        else if (px < 13'd320)  px_color = 24'h80_80_80;
        ...  // 共 8 段
    end
endfunction
```

这里曾经写错过一次

首版用位切片判竖条：`case (px[12:8])`。但 160 不是 2 的幂，位切片得到的分段与 160px 等宽完全不符。

修正：改用比较链（`px < 160 / px < 320 / ...`）。综合器会把比较链优化成并联比较器，资源几乎不变。

11.4 4 分频节拍（关键设计）

```
reg [1:0] pres;
wire do_send = (pres == 2'd0) & (~GAP_EN | ...);

always @(posedge I_clk) begin
    pres <= pres + 2'd1;          // 0,1,2,3,0,1,2,3...
    if (do_send) begin
        // 输出一个 beat（4 像素）
    end
end
```

为什么必须 4 分频？ 720p60 的像素率 55.3Mpx/s，在 74.25MHz 时钟下 beat 率只有 13.8M/s，即**每 4 拍 1 个 beat**。如果 TPG 满速发，下游 FIFO 立刻溢出（这就是上一节那个真 bug）。

踩过的坑：复位遗漏

首版 `pres` 忘了写进复位分支，上电后是 X（未知态），`pres == 2'd0` 永远不成立 → TPG 一个像素都不输出。

教训：每个寄存器都必须有复位值。这是新手最常犯的错误，而且现象往往是”完全没输出”，很难联想到复位。

11.5 帧同步标志

```
O_tuser <= (beat_x == 0) && (y == 0);          // 整帧仅一拍为 1
O_tlast <= (beat_x == BEATS_PER_LINE - 1);     // 每行最后一拍
```

`tuser` 只在”第 0 行第 0 拍”为 1——即一帧的最开头；`tlast` 在每行第 319 拍为 1——即一行的最右端。

11.6 LFSR 随机空闲（可选，锻炼鲁棒性）

GAP_EN=1 时用线性反馈移位寄存器产生约 7% 概率的空闲拍，模拟真实总线的随机间隙：

```
lfsr <= {lfsr[14:0], lfsr[15] ^ lfsr[13] ^ lfsr[12] ^ lfsr[10]};
wire do_send = (pres == 2'd0) && (~GAP_EN | (lfsr[0] | (lfsr[7:0] == 8'h00)));
```

12 模块详解：pulse_sync（单 bit 脉冲跨域）

12.1 问题

一个 74.25MHz 域的”单拍脉冲”（例如每帧一次的帧拍），要传给 50MHz 域。直接连一根线会出两个问题：

- 50MHz 采样时可能采到脉冲的**边沿** → 亚稳态
- 74.25MHz 的单拍脉冲只有 13.5ns 宽，50MHz 周期是 20ns——可能被**完全错过**（一次都采不到）

12.2 解决方案：toggle + 边沿还原

```
// 源域：每来一个脉冲，翻转一次电平
reg toggle_src;
always @(posedge I_clk_src or negedge I_rst_src_n) begin
    if (!I_rst_src_n)     toggle_src <= 1'b0;
    else if (I_pulse_src) toggle_src <= ~toggle_src;
end

// 目标域：3 级同步 + 边沿检测
reg [2:0] sync_dst;
always @(posedge I_clk_dst or negedge I_rst_dst_n) begin
    if (!I_rst_dst_n) sync_dst <= 3'b000;
    else              sync_dst <= {sync_dst[1:0], toggle_src};
end

assign O_pulse_dst = sync_dst[1] ^ sync_dst[2];   // 电平变化 = 一个脉冲
```

原理 电平（持续高或持续低）比单拍脉冲”宽”，容易安全采样。目标域检测到电平发生翻转（sync_dst[1] != sync_dst[2]），就知道”源域发了一个脉冲”，于是产生自己域内的一个单拍脉冲。

约束条件（必须满足）

源脉冲之间的间隔必须大于目标域 3 个时钟周期，否则两次翻转来不及被同步，会漏脉冲。

本项目用于”每帧一次”的帧拍——帧间隔 16.7ms，远大于 60ns，绝对安全。

13 模块详解：cdc_4phase（多 bit 数据跨域）

13.1 问题

目标包（68bit，拓展后 135bit）要从检测域传到显示域/云台域。多 bit 不能像单 bit 那样简单同步——因为每根线的延迟略有差异，接收方可能采到”新旧混合”的值（例如高 8 位是新坐标、低 8 位是旧坐标）。

13.2 解决方案：4 相握手

```
源域                                          目标域
———                                           ———
1. 数据放总线上（保持不动）
2. 翻转 req  ————————————————————————————> 3. 检测到 req 变化
                                              4. 锁存数据（此时数据已稳定）
                                              5. 翻转 ack
6. 检测到 ack 变化 <————————————————————————┘
7. 解除 busy，可以发下一包
```

为什么能保证数据正确？ 关键是**数据在 req 翻转之前就已稳定，且整个握手期间保持不变**。目标域在检测到 req 变化后的任意时刻采样，都采到完整的新数据。

类比：寄快递签收

你把包裹放门口（数据），贴一张”已放件”标签（req 翻转）。快递员取走包裹，贴一张”已取件”回执（ack 翻转）。你看到回执才放下一个包裹——绝不会出现”包裹只放了一半就被取走”。

13.3 关键实现与一个真实 bug

```
1  // 源域：busy 的解除条件
2  if (!busy) begin
3      if (I_src_load) begin
4          data_hold <= I_src_data;    // 1. 先锁存数据
5          req       <= ~req;          // 2. 再翻转 req
6          busy      <= 1'b1;
7      end
8  end else if (ack_2ff == req) begin
9      busy <= 1'b0;                   // 3. 用" 电平比较" 判断对方已应答
10 end
```

真实 bug：TB 驱动的同沿竞态

首版仿真发现**每隔一包丢一包**（1,3,5,7…收到，2,4,6 丢失）。

根因不在 RTL，在测试平台：`wait(!busy)` 在 busy 被置位的**同一个时钟沿**就通过了（因为非阻塞赋值此时还未生效），于是第二个包在 DUT 仍忙时发出，被丢弃。

修正：驱动侧改为”先等 busy 上升 → 再等 busy 下降”的稳健时序。

教训：跨域模块的 TB 必须显式等待握手完成，不能假设一个时钟沿就够。定位这个 bug 用了逐拍波形追踪（打印 req/busy/ack 的每拍值），最后靠”数据序列号”看出是隔包丢失——如果是随机数据根本看不出规律。

13.4 验证

`tb_cdc4phase` 用两个异步时钟（周期 14ns 与 20ns，无固定相位关系），发送 200 个随机 23bit 数据，逐个校验完整性。全部通过。

14 仿真专用模块

14.1 sim_video_src（文件回放源）

从 `chain_beats.hex`（96bit 拍数据）与 `chain_flags.hex`（标志位）读入 Python 生成的确定性场景，按帧重复播放。

踩过的坑：readmemh 的位宽限制

首版 flags 文件每行写两个字符（如 `"10"` 表示 tuser=1, tlast=0），但 TB 里声明的是 `reg [1:0] flags`——只允许 **1 位**十六进制数字，readmemh 直接截断，所有标志位全毁（tuser/tlast 永远为 0）。

修正：改成单字符格式（值 0~3：2=tuser、1=tlast、3= 两者）。

教训：hex 向量文件的字段宽度 = 向量位宽/4，宁可浪费一位也不要溢出。

14.2 frame_dump24（帧转储）

把 vpix 流（1 像素/拍）写成 hex 文件，每行一个像素的 RGB888：

```
$fwrite(fd, "%02X%02X%02X\n", I_data[23:16], I_data[15:8], I_data[7:0]);
```

小陷阱：ModelSim 的%X 输出小写

Python 比对时要注意：`c0c0c0` 与 `C0C0C0` 是同一个颜色。`run_all.py` 里统一做了 `.upper()` 归一化，避免这类假失配。

15 测试平台设计

15.1 四层验证策略

层次	方法	能抓到的错误
模块级	随机向量 + 逐位比对	算术错误、标志错位
帧级	整帧 921,600 像素比对	几何错误、边界处理错误
协议级	异步双钟 + 随机数据	跨域丢包、亚稳态
系统级	端到端场景	模块间接口不匹配

15.2 tb_ser96to24 的关键设计：唯一数据编码

```
1 // 期望队列：把输入展开后逐像素压入
2 for (k = 0; k < 4; k = k + 1) begin
3     exp_d[push_n + k] = 24'h100000 * f + b * 4 + k;  // 唯一编码
4     exp_s[push_n + k] = (b == 0) && (k == 0);          // 帧首只在第 1 像素
5     exp_e[push_n + k] = (k == 3) && ((b % BEATS) == BEATS-1); // 行尾只在第 4 像素
6 end
```

设计要点：数据用唯一编码 第 f 帧、第 b 拍、第 k 个像素的值 = `0x100000*f + b*4 + k`。每个像素值全网唯一——出错时一眼看出是**哪个像素**错位了，而不只是知道" 有错误"。这比随机数据

强大得多。

15.3 tb_tpg 的双重验证

1. **几何断言**：每帧必须恰好 1 次 tuser、720 次 tlast、921,600 个像素
2. **帧转储比对**：写出的 `tpg_act_0.hex` 与 Python 生成的 `tpg_exp.hex` 逐像素一致

Python 侧的期望帧用**同一个颜色表**生成（`gen_vectors.py::tpg_frame`），保证" 设计意图" 与"RTL 实现" 完全对齐——这是自动化验证的前提。

15.4 tb_cdc4phase 的时序陷阱

```
// 正确的驱动时序（修正同沿竞态后）
data_v = WIDTH'($urandom());
s_data <= data_v;
exp_q[wp] <= data_v;
s_load <= 1'b1;
@(posedge clk_s);
s_load <= 1'b0;
@(posedge clk_s);            // 等一拍，让 busy 有条件拉高
wait (s_busy === 1'b1);      // 显式等 busy 拉高（受理）
wait (s_busy === 1'b0);      // 显式等握手完成
sent = sent + 1;
```

16 验证结果

16.1 回归输出

```
$ python tools/run_all.py

--- tb_ser96to24 ...
    ==> PASS
--- tb_tpg ...
    ==> PASS
--- tb_pulse_sync ...
    ==> PASS
--- tb_cdc4phase ...
    ==> PASS
...
==== 汇总: 18/18 PASS ====    ← 当前仓库全量（Step1 阶段为 15 项）
```

16.2 Step1 四项测试细节

测试	验证内容	结果
tb_ser96to24	随机空拍 + 3 帧，队列自检（数据/sof/eol 逐像素）	PASS
tb_tpg	几何计数断言 + 921,600 像素逐像素比对	PASS
tb_pulse_sync	异步双钟 100 脉冲 1:1 传递	PASS
tb_cdc4phase	异步双钟 200 个随机 23bit 数据完整性	PASS

16.3 Step1 阶段抓到的 5 个真缺陷

#	现象	根因与修复
1	串化器溢出（cnt=13）	源满速发 beat（应为 1/4 占空）→ 全体加 4 分频
2	CDC 隔包丢	TB `wait(!busy)` 同沿竞态 → 显式等握手完成
3	TPG 无输出	`pres` 漏复位（上电 X）→ 补复位分支
4	flags 全毁	hex 字段 2 字符超过 2bit 向量限制 → 改单字符
5	TB 编译错	缺 `` `timescale `` 指令 → 补上

这 5 个缺陷的价值

全部 5 个都在**仿真阶段**发现并修复，没有一个留到上板。

这正是" 金标准 + 逐位比对 + 严格 TB" 方法论的价值——上板调试一次的成本（综合 20 分钟 + 下载 + 示波器测量 + 重新综合）远高于在仿真里跑几十秒定位问题。

17 上板挂载说明

17.1 在官方例程中的位置

Step1 的模块**不改变**官方 ISP→DDR→HDMI 主线，是纯旁路分支：

```
isp_top 输出（96bit AXI-S）
    │
    ├──> data_96bit_to_128bit ──> DDR2 ──> HDMI（主线，原封不动）
    │
    └──> detect_top（视觉分支）
            │
            ├── ser96to24 ──> 1ppc ──> HSV/阈值/平滑/质心（Step2）
            │
            └── 直通输出（与输入逐拍一致，TB 已断言 passthru_mism=0）
```

17.2 挂载步骤（给上板组）

1. 在 `design_top_wrapper.v` 中找到 `isp_top` 的实例
2. 把它的输出（`O_tdata / O_tvalid / O_tuser / O_tlast`）**并线**接到 `detect_top` 的输入（并线，不是串联）
3. `detect_top` 的 `O_tready` 悬空或接常量 1（模块内部已固定为 1）
4. 综合：确认无新警告
5. 上板：用 ChipWatcher 抓 `t_valid` 与 `t_tick`，确认每帧有脉冲

17.3 资源占用

模块	资源估算	说明
ser96to24	$<$ 200 LUT + 8×26bit 分布式 RAM	FIFO 很小
tpg_720p	$<$ 300 LUT	只有计数器与比较器
pulse_sync	$<$ 10 LUT	4 个触发器
cdc_4phase	$<$ 100 LUT	握手逻辑
Step1 合计	**$<$ 610 LUT**	约占 PH1P35 的 1.6%

为什么资源这么少

因为 Step1 只做” 搬运”——没有乘法、没有除法、没有大缓存。

真正的资源消耗在后面：HSV 查表（256×12 ROM）、行缓存（2×1280bit）、运动检测帧缓存（115KB ERAM）。

18 总结与下一步

18.1 Step1 学到了什么

- Verilog 最小语法集（8 个结构 + 3 条铁律）
- 视频流三路标（valid/sof/eol）与帧结构
- FIFO 的原理和实现（写指针/读指针/计数器）
- 串化器的思路（4 像素一包 → 1 像素一拍）
- 测试图案发生器的思路（计数器驱动查色表）
- CDC 的必要性和两种安全桥

18.2 下一步学什么

- **Step2 颜色识别与质心**：在像素流上做实时图像处理
  - RGB→HSV 颜色空间变换（查表 + 移位代替除法）
  - 阈值分割（” 这个像素是不是目标颜色？”）
  - 3×3 平滑（去噪）
  - 质心计算（” 目标在画面哪里？”）
- 对应文档：`03_vision/step2/02_ 技术文档/`
- 对应金标准：`tools/golden_model.py`